

I want you to read over this assignment, and then read over every file in this repository. Then tell me any questions that you have about it.

Assignment 2: Web Crawler

In this project, you are going to implement the core of a Web crawler, and then you are going to crawl the **following URLs (to be considered as domains for the purposes of this assignment) and paths:**

- *.ics.uci.edu/*
- *.cs.uci.edu/*
- *.informatics.uci.edu/*
- *.stat.uci.edu/*

As a concrete deliverable of this project, besides the code itself, you must submit a report containing answers to the following questions:

1. How many *unique pages* did you find? **Uniqueness for the purposes of this assignment is ONLY established by the URL, but discarding the fragment part.** So, for example, *http://www.ics.uci.edu#aaa* and *http://www.ics.uci.edu#bbb* are the same URL. Even if you implement additional methods for textual similarity detection, please keep considering the above definition of unique pages for the purposes of counting the *unique pages* in this assignment.
2. What is the longest page in terms of the number of words? (*HTML markup doesn't count as words*)
3. What are the 50 most common words in the entire set of pages **crawled under these domains** ? (**Ignore English stop words**, which can be found, for example, [here \(https://www.ranks.nl/stopwords\)](https://www.ranks.nl/stopwords))
4. [Links to an external site.](#)
5.) Submit the list of common words ordered by frequency.
6. How many subdomains did you find in the uci.edu domain? Submit the list of subdomains ordered alphabetically and the number of unique pages detected in each subdomain. The content of this list should be lines containing *subdomain, number*, for example:
vision.ics.uci.edu, 10 (not the actual number here)

What to submit: a zip file containing your modified crawler code and the report.

Grader meetings: this project requires a meeting of all members of your group with one of the TAs/Readers, where all of you will be asked questions about your crawler -- your code and the operation of the crawler. These meetings will occur a few days after the submission deadline. Instructions will be sent at the time.

Specifications

To get started, fork or get the crawler code from
<https://github.com/Mondego/spacetime-crawler4py>

[Links to an external site.](#)

Read the instructions in the README.md file up to, and including the section "Execution". This is enough to implement the simple crawler for this project. In short, this is the minimum amount of work that you need to do:

1. Install the dependencies
2. Set the USERAGENT variable in Config.ini so that it contains all **students' IDs separated by a comma (the numbers! e.g. IR US26 123123213, 12312312, 123123)** of the group members. **If you fail to do this properly, your crawler will not exist in the server's log, which will put your grade for this project at risk. NOTE FROM STUDENT: THIS IS ALREADY DONE**
3. **(This is the meat of the crawler)** Implement the scraper function in scraper.py. The scraper function receives a URL and corresponding Web response (for example, the first one will be "http://www.ics.uci.edu" and the Web response will contain the page itself). Your task is to parse the Web response, extract enough information from the page (*if it's a valid page*) so to be able to answer the questions for the report, and finally, return the list of URLs "scrapped" from that page. Some important notes:
 1. **Make sure to return only URLs that are within the domains and paths mentioned above!** (see `is_valid` function in scraper.py -- you need to change it)
 2. Make sure to defragment the URLs, i.e. remove the fragment part.
 3. You can use whatever libraries make your life easier to parse things. Optional dependencies you might want to look at: BeautifulSoup, lxml (nudge, nudge, wink, wink!)
 4. Optionally, in the scraper function, you can also save the URL and the web page on your local disk.

5. See Crawler Details (located below)
4. Run the crawler from your laptop/desktop or from an **ICS openlab machine** (you can use either the classical `ssh&scp` to `openlab.ics.uci.edu` or you can use the web interface `hub.ics.uci.edu` from your browser; I would recommend you to use `ssh`, such that you will learn a skill that will be probably important for the rest of your professional life... note that to install any software in machines that you do not own or that you are authorized to `sudo`, you need to install them to your user folder, and in `pip/pip3` you need to use the `--user` option to do so). Note that this will take several hours, possibly a day! It may even never end if you are not careful with your implementation! Note that you need to be inside the campus network, or you won't be able to crawl. If your computer is outside UCI, use the VPN.
5. **Monitor what your crawler is doing.** If you see it trapped in a Web trap, or malfunctioning in any way, stop it, fix the problem in the code, and restart it. Sometimes, you may need to restart from scratch. In that case, delete the frontier file (`frontier.shelve`), or move it to a backup location, before restarting the crawler.

Crawler Behavior Requirements

In this project. we are looking for **text** in Web pages so that we can search it later on. The following is a list of what a "correct crawl" entails in this context:

- Honor the politeness delay for each site
- Crawl all pages with high textual information content
- Detect and avoid infinite traps
- Detect and avoid sets of similar pages with no information
- Detect and avoid dead URLs that return a 200 status but no data ([click here to see what the different HTTP status codes mean](#))
- [Links to an external site.](#)
-)
- Detect and avoid crawling very large files, especially if they have low information value

For most of these requirements, the only way you can detect these problems is by first monitoring where your crawler is going, and then adjusting its behavior in order to stay away from problematic pages.

Test Period and Deployment Period

Due to the nature of this project, the time allocated to it is divided into two parts:

Test: until May 1, 8:00pm. During this time, your crawler can make all sorts of mistakes -- try to crawl outside allowed domains, be impolite, etc. No penalties if you are figuring things out; but penalties if you knock down the server on purpose (i.e. *removing* politeness delays). Do not wait until the last minute to start working on your assignment; this assignment requires you to experiment significantly, and you will probably not have time to finish it if you start it late.

Deployment: from May 1, 9:00pm until May 7, 8:00am. This is the real crawl. During this time, your crawler is expected to behave correctly. **Even if you finish your project earlier, you must operate your crawler during this time period, but you must not restart the crawl more than three times during this period** (unless there is a server issue; *note that they are all recorded*). **You must submit your assignment on Canvas by the due date/time.**

Late assignments: For late assignments, you **cannot** disrespect the crawling policy during the normal deployment period (*you can only crawl three times during deployment, no more!* **MAKE SURE TO DEVELOP AND TEST YOUR CRAWLER DURING THE TEST PERIOD**). Then you can have from May 7, 9:00am until May 14, 8:00am as an additional Test period, and from May 14, 9:00am until May 19, 8:00am as an additional Deployment period. **You need to submit your late assignment in Canvas with an automatic penalty of 25% of the grade by the deadline, and no submission will be accepted after May 19, not even for reduced points.**

Note: The cache server may die for a few hours during these periods due to loads created by impolite crawlers. We will be monitoring closely the server, and it will be back online after (at most) **~8h hours during the Test period, and after (at most) ~4h during the Deployment period (unless it happens to die during the night - 10:00pm till ~7:00am).** Make sure to abide by politeness rules and respect to your colleagues, especially if you are trying to implement a multithreaded crawler.

Grading criteria

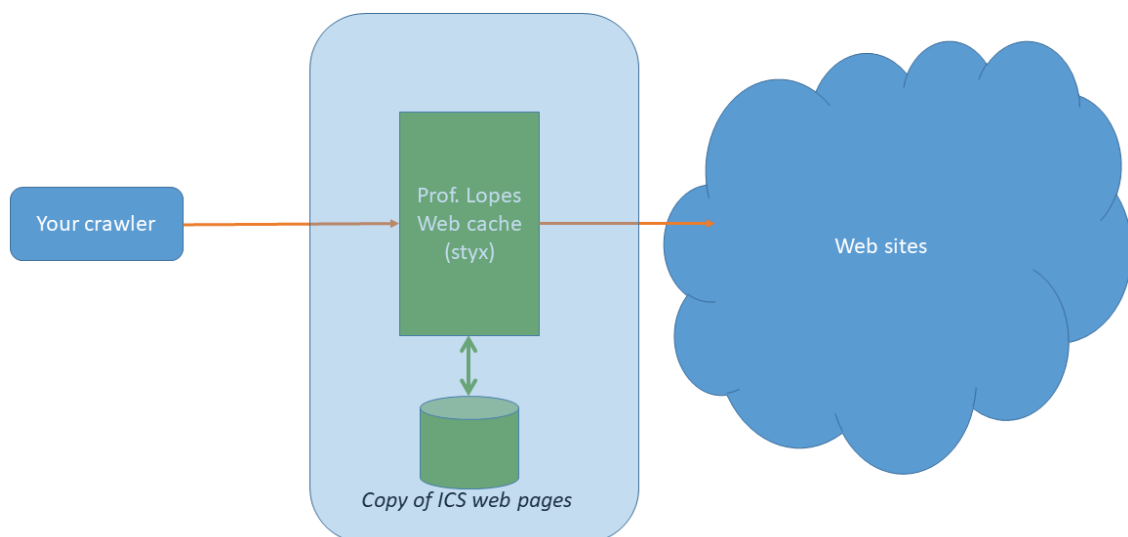
1. Are the analytics in your report within the expected range? (10%)
2. Did your crawler operate correctly? -- we'll check our logs (~45%)
 1. Does it exist in Prof. Lopes' Web cache server logs? **(if it's not in the ICS logs, it didn't happen: you will get 0)**
 2. Was it polite? (penalties for impolite crawlers)
 3. Did you crawl ALL domains and paths mentioned in the spec? (penalties for missing domains and paths)

4. Did it crawl **ONLY** the domains and paths mentioned in the spec? (penalties for attempts to crawl outside)
 5. Did it avoid traps? (penalties for falling in traps)
 6. Did it avoid sets of pages with low information value? (penalties for crawling useless families of pages - **you must decide and discuss within your group on a reasonable definition for a low information value page and be able to defend it during the interview with the TAs**)
3. **Are you able to answer and justify the questions about your code and the operation of your crawler?** (~45%)

Technical Details

In order not to disrupt the ICS network, your crawlers use a Web cache that is specifically designed for this project, and that runs on one of Prof. Crista Lopes servers. The following picture illustrates the architecture of this project:

Architecture of Web Crawling Project



If you use the crawler code properly, the cache is largely invisible to you. But you should be aware that it is there. At certain points, you may receive errors that are specifically sent by the cache server to your crawler, when your crawler is doing something it

shouldn't -- they are in the 600 range of the status code. **DO NOT ATTEMPT TO BYPASS THE CACHE!** -- that may seriously disrupt the ICS network, which may pose additional problems during this already complex quarter.

Here are the HTTP status codes that your crawler may receive from the cache server:

- 600: Request Malformed
- 601: Download Exception {error}
- 602: Spacetime Server Failure
- 603: Scheme has to be either http or https
- 604: Domain must be within spec
- 605: Not an appropriate file extension
- 606: Exception in parsing url
- 607: Content too big. {resp.headers['content-length']}
- 608: Denied by domain robot rules

You may ignore some of them, but not all.

Crawler Details

This is the meat of what you need to change/do in the crawler code:

```
import re
from urllib.parse import urlparse

def scraper(url, resp):
    links = extract_next_links(url, resp)
    return [link for link in links if is_valid(link)]

def extract_next_links(url, resp):
    # Implementation required.
    return list()

def is_valid(url):
    try:
        parsed = urlparse(url)
        if parsed.scheme not in set(["http", "https"]):
            return False
```

```

        return not re.match(
            r"*\.(css|js|bmp|gif|jpe?g|ico"
            + r"|png|tiff?|mid|mp2|mp3|mp4"
            + r"|wav|avi|mov|mpeg|ram|m4v|mkv|ogg|ogv|pdf"
            + r"|ps|eps|tex|ppt|pptx|doc|docx|xls|xlsx|names"
            + r"|data|dat|exe|bz2|tar|msi|bin|7z|psd|dmg|iso"
            + r"|epub|dll|cnf|tgz|sha1"
            + r"|thmx|mso|arff|rtf|jar|csv"
            + r"|rm|smil|wmv|swf|wma|zip|rar|gz)$",
        parsed.path.lower())

    except TypeError:
        print ("TypeError for ", parsed)
        raise

```

- `extract_next_links(url, resp)` gives you a URL and the response from the server (i.e. the page on that URL). You are supposed to take the response and process it in order to extract the hyperlinks from it, and return them in a list of strings. The response has the following fields available:
`resp.url`: the actual url of the page
`resp.status`: the status code returned by the server. 200 is OK, you got the page. Other numbers mean that there was some kind of problem.
`resp.error`: when status is not 200, you can check the error here, if needed.
`resp.raw_response`: this is where the page actually is. More specifically, the `raw_response` has two parts:
 - `resp.raw_response.url`: the url, again
 - `resp.raw_response.content`: the content of the page!
- `is_valid(url)` gives you a URL and you are supposed to decide whether to crawl it or not. If you decide to crawl it, return True; otherwise return False. The function already has a few things for which it returns False. You will need to add a few more, as you encounter collections of pages that may look like traps.

Running/Restarting the crawler

If you need to stop the crawler, press Ctrl-C. If you need to restart it from where it left off, simply run it again. If you need to start a fresh crawl from scratch, first delete the file `frontier.shelve`, and then run the crawler again.

Inspecting the Log

Your crawler's log file is under Logs/Worker.log. Make sure that you inspect that Log file in order to understand what happened during the test crawls.

Server Cache status codes

These are all the cache server error codes:

- 600: Request Malformed
- 601: Download Exception {error}
- 602: Spacetime Server Failure
- 603: Scheme has to be either http or https
- 604: Domain must be within spec
- 605: Not an appropriate file extension
- 606: Exception in parsing url
- 607: Content too big. {resp.headers['content-length']}
- 608: Denied by domain robot rules

You may ignore some of them, but not all.